

KRANTHI KUMAR

AI Engineer / Software Engineer

(361) 253-1951 | Charlotte, NC | kranthiyadav1997@gmail.com | [LinkedIn](#)

TECHNICAL SKILLS

Languages: Python, Java, SQL, Node.js, C++

AI/ML & Agents: LangGraph, CrewAI, MCP, Multi-Agent Orchestration, RAG, Vector Embeddings, Chunking, BM25, pgvector, RAGAS, LoRA/PEFT, PyTorch, Hugging Face

Model Ops & Security: Model Routing, Agent Observability, PII Masking, DLP, Pydantic Validation, Prompt Engineering, Guardrails, OpenTelemetry, Prometheus, Grafana

Data Engineering: ETL/ELT Pipelines, Airflow (DAGs), Kafka, Spark, SSIS, CDC, Data Lineage, Data Cleaning, Tableau, Power BI

Cloud & DevOps: AWS (S3, Glue, Lambda, Redshift, EMR, CloudWatch), Azure (Data Factory, Blob Storage), Docker, Kubernetes, CI/CD

Databases & APIs: PostgreSQL, Oracle, Redshift, DynamoDB, Redis (Agent Memory) | REST (FastAPI, Flask), GraphQL, gRPC

PROFESSIONAL EXPERIENCE

Cloud Nest IT – AI Engineer

Charlotte, NC | Jul 2025 – Present

- Own two agents in a five-agent financial advisory platform built on LangGraph: a Planning Agent that decomposes advisor queries and routes them through conditional gates to the right downstream agent, and a Memory Agent (Redis-backed) that tracks conversation state across sessions.
- Orchestrate the five-agent workflow using LangGraph chains with conditional edges and gate nodes — the Planning Agent selects which specialist agents to invoke based on query type, so simple lookups skip expensive reasoning agents entirely.
- Built the RAG retrieval pipeline: chunked ~50k compliance PDFs into overlapping 512-token segments, embedded them with sentence-transformers into pgvector, and added BM25 reranking — cutting retrieval latency from 12s to under 3s with a RAGAS faithfulness score of 0.91.
- Trained a LoRA-fine-tuned routing classifier (PyTorch, Hugging Face) that sends simple lookups to a smaller LLM and complex reasoning to GPT-4o, cutting monthly inference cost by 30% across ~10k daily requests.
- Added a PII-masking and DLP layer (regex + NER + Pydantic schema validation) between agents and the API response — rejected 4% of raw LLM outputs containing hallucinated or sensitive data before they reached ~200 advisors.
- Instrumented all five agents with OpenTelemetry distributed tracing (span per tool call and LLM invocation) and Prometheus counters for token usage and latency p95/p99, so the team traces failed responses across the full agent chain.
- Evaluated RAG pipeline quality with RAGAS (faithfulness, answer relevancy, context precision) and wrote pytest integration tests replaying recorded agent traces in CI on every GitHub PR, with data-contract checks on the vector store ETL pipeline.

Aviva – Software Engineer- Data

India | Jan 2023 – Aug 2023

- Built the backend for policy and claims processing: 8 REST API endpoints in Python (FastAPI) and 3 in Node.js, serving 30+ underwriters who previously waited 2–3 hours for data that now returned in under 200ms.
- Replaced a nightly CSV-over-SFTP batch transfer with a GraphQL API layer (queries, mutations, schema definitions), so two downstream reporting systems received claims updates in near real time instead of 24 hours behind.
- Profiled PostgreSQL during peak-hour connection exhaustion: redesigned the schema (14 tables, 6 views), added composite indexes on the 5 most-queried joins, and introduced PgBouncer — cutting p95 query time from 800ms to 120ms.
- Built a data-cleaning layer in the ingestion pipeline that caught and quarantined ~6% of incoming broker records (malformed dates, duplicate policy IDs, null-filled fields) before they reached the claims database.
- Containerized all 11 microservices in Docker, deployed on Kubernetes with liveness probes and rolling updates — resolving a recurring deployment crash caused by config mismatches between staging and production.

Amazon – Software Engineer- ML

Hyderabad, India | Nov 2021 – Dec 2022

- Worked on the AWS HoneyCode platform team building Python and Java backend services — resolved a critical memory leak in the event-processing pipeline that caused 3–4 OOM crashes per week, stabilizing uptime to 99.9%.
- Built ETL pipelines on AWS (S3 → Glue → Redshift) extracting ~500k usage and billing records per month across 8 source feeds, with deduplication and schema normalization before loading into a Redshift warehouse.
- Replaced fragile cron-based triggers with AWS Lambda on S3 event notifications and set up 15+ CloudWatch alarms for latency, error rate, and queue depth — reducing failure detection from next-morning to under 5 minutes.
- Migrated a single-threaded Python ingestion script to Kafka (6 topics, 18 partitions) and Spark on a 5-node EMR cluster with dead-letter queues — throughput went from ~2k to ~50k records/hour.
- Designed a star schema in Redshift (3 fact tables, 8 dimensions) unifying DynamoDB (4 collections) and RDS (12 tables), resolving data-type mismatches and key collisions that caused duplicate billing entries.

- Wrote 20+ SQL stored procedures and views in Redshift for data-quality checks catching ~3% bad records per run — including a timezone-conversion bug that had silently miscalculated billing for 3 months.
- Used Git/GitHub for version control, pull requests, and code reviews; documented data flows, Glue job configurations, and the Redshift schema for the engineering team.

Cyient – Data Engineer

Hyderabad, India | Mar 2019 – Oct 2021

- Inherited a manual process where an analyst copy-pasted CSV rows into PostgreSQL by hand weekly — wrote Python ETL scripts extracting ~200k records from 6 Oracle tables and CSV dumps, validated and loaded them in under 40 minutes.
- Built 5 SSIS packages moving inspection data between SQL Server and PostgreSQL nightly, with row-count reconciliation — caught a silent data-truncation issue in the first week that had been dropping ~2% of records for months.
- Built 8 REST API endpoints in Python (Flask) for an inspection portal that timed out on large queries — added paginated SQL, composite indexes, and query-plan tuning, bringing p95 from 4+ seconds to under 200ms at ~2k requests/day.
- Migrated manual FTP uploads to Azure Data Factory pipelines orchestrating Oracle → Azure SQL Database → PostgreSQL, with schema-drift detection so new source columns did not silently break downstream loads.
- Built 3 Tableau dashboards for inspection completion rates, defect trends by component type, and SLA compliance — replacing a Friday spreadsheet that project managers spent a full day compiling.
- Debugged a nightly batch job timing out on 80k records: traced it to a nested-loop join on an unindexed column, replaced with a hash-map lookup in Python — run time dropped from 4+ hours to under 20 minutes.
- Documented end-to-end data lineage from Oracle/CSV sources through ETL scripts and SSIS packages to PostgreSQL and the Flask API, with a runbook for each failure mode the team had encountered.

PROJECTS

Mini vLLM – Toy LLM Inference Engine

C++, CUDA, PyTorch

- Built a paged KV-cache allocator and continuous-batching scheduler in C++ with CUDA kernels to study how serving engines keep GPUs busy — validated outputs against PyTorch and benchmarked throughput gains over a naive loop.

EDUCATION

Texas A&M University – Corpus Christi – M.S. Computer Science

Aug 2023 – May 2025